ӨГӨГДЛИЙН САНГИЙН УДИРДЛАГА

F.ITM237

ЛАБОРАТОРИЙН УДИРДАМЖ №4-5

Өгөгдлийн сангийн аюулгүй байдал: Хэрэглэгч, Роль, Эрх, Аудит

Ажиллах орчин: Microsoft SQL Server | **Нийт оноо: 3 оноо**

Мэдээллийн технологийн тэнхим | С.Өлзийбаяр (Ph.D)

1 ЛАБОРАТОРИЙН ЗОРИЛГО

5-р долоо хоногийн лекцийн агуулга болох өгөгдлийн сангийн аюулгүй байдлыг практикт хэрэгжүүлэх ур чадварыг эзэмшихэд чиглэнэ. Оюутнууд дараах чадваруудыг эзэмшинэ:

- SQL Server дээр LOGIN болон USER үүсгэж, серверийн болон өгөгдлийн сангийн түвшний хандалтыг тохируулах
- ROLE үүсгэж хэрэглэгчдэд үүрэг хуваарилах — RBAC (Role-Based Access Control) аргачлалыг хэрэглэх
- GRANT, REVOKE, DENY командуудын ялгааг практикт харуулах
- Хамгийн бага эрхийн зарчмыг (Least Privilege Principle) баримтлан олон хэрэглэгч тохируулах
- SQL Injection халдлагаас хамгаалах Parameterized Query ашиглах
- SQL Server Audit тохируулж, нэвтрэлт болон өөрчлөлтийн логийг хянах

2 ШААРДЛАГАТАЙ ПРОГРАМ ХАНГАМЖ

- Microsoft SQL Server 2019 эсвэл түүнээс дээш хувилбар
- SQL Server Management Studio (SSMS) эсвэл Azure Data Studio
- Lab02-ын UniversityDB өгөгдлийн сан (Department, Student хүснэгтүүд)
- SQL Server Authentication горим идэвхтэй байх ёстой — Mixed Mode Authentication

3 ОНОЛЫН ҮЙЛ

3.1 CIA Triad — Аюулгүй байдлын гурван тулгуур

Команд / Ойлголт	Тайлбар
`Нууцлал (C)`	Зөвхөн эрх бүхий хэрэглэгч мэдээлэлд хандах. Шифрлэлт, хандалтын хяналт ашиглана
`Бүрэн бүтэн байдал (I)`	Өгөгдлийг зөвшөөрөлгүйгээр өөрчлөхөөс сэргийлэх. CHECK, FOREIGN KEY, аудит ашиглана
`Хүртээмж (A)`	Зөвшөөрөгдсөн хэрэглэгч дуртай үедээ хандах боломжтой байх. DoS халдлагаас хамгаалах

3.2 Login ба User — Ялгаа

Команд / Ойлголт	Тайлбар
`LOGIN`	Серверийн түвшинд үүсдэг. SQL Server-т нэвтрэх 'хаалгын түлхүүр'. CREATE LOGIN
`USER`	Өгөгдлийн сангийн түвшинд үүсдэг. Тодорхой DB-д хандах 'өрөөний түлхүүр'. CREATE USER
`ROLE`	Эрхүүдийг бүлэглэсэн объект. Олон хэрэглэгчид нэгэн зэрэг эрх олгоход ашиглана
`GRANT`	Тодорхой эрхийг олгоно. Хэрэглэгч болон үүрэгт хэрэглэнэ
`REVOKE`	Олгосон эрхийг цуцалдаг. Хэрэглэгч өөр замаар эрх авч болно — зөөлөн хандлага
`DENY`	Эрхийг хатуу хориглоно. GRANT байсан ч DENY давамгайлна — хамгийн хүчтэй хориг

3.3 SQL Injection ба Parameterized Query

SQL Injection халдлагад 'OR 1=1' гэх мэт нэмэлт SQL код оруулан баталгаажуулалтыг алгасдаг. Хамгаалах гол арга бол параметрт суурилсан асуулга (Prepared Statement) ашиглах юм — параметр нь SQL кодын хэсэг биш, харин зөвхөн өгөгдөл хэлбэрээр дамжина.

3.4 SQL Server Audit — Үндсэн тойм

Команд / Ойлголт	Тайлбар
`SERVER AUDIT`	Аудитын объект үүсгэж, лог файл хадгалах газрыг тохируулна
`AUDIT SPEC`	Аудитлах үйлдлүүд (INSERT, UPDATE, DELETE, SELECT) тодорхойлно
`STATE = ON/OFF`	Аудитыг идэвхжүүлэх / зогсооход ашиглана
`fn_get_audit_file`	Аудитын лог файлаас мэдээлэл унших системийн функц

0 ӨГӨГДЛИЙН БЭЛТГЭЛ (Лаб эхлэхээс өмнө)

Lab02-оос үүсгэсэн UniversityDB өгөгдлийн санд холбогдоно. Энэ лабораторид аудитын лог хадгалах хавтас болон туршилтын нууц хүснэгт нэмж бэлтгэнэ.

```
USE UniversityDB;
GO

-- Windows Explorer дээр C:\AuditLogs\ хавтас үүсгэнэ (аудитад шаардлагатай)

-- Туршилтын нууц хүснэгт үүсгэх
CREATE TABLE SecretData (
    id         INT PRIMARY KEY IDENTITY(1,1),
    info       NVARCHAR(200),
    created_at DATETIME DEFAULT GETDATE()
);
```

```
INSERT INTO SecretData (info) VALUES
    (N'Нууц мэдээлэл 1 - зөвхөн admin харна'),
    (N'Нууц мэдээлэл 2 - зөвхөн admin харна'),
    (N'Нийтийн мэдээлэл - бүх хэрэглэгч харна');

-- Шалгах
SELECT COUNT(*) AS StudentCount  FROM Student;
SELECT COUNT(*) AS DeptCount     FROM Department;
SELECT COUNT(*) AS SecretCount   FROM SecretData;
```

4 ПРАКТИК ДААЛГАВАР

Даалгавар 1: Login болон User үүсгэх	0.8 оноо

1а. SQL Server Login үүсгэх (0.4 оноо)

UniversityDB-д хандах гурван хэрэглэгчийн серверийн Login үүсгэнэ. Нууц үг нь дор хаяж 8 тэмдэгт, том, жижиг үсэг, тоо агуулсан байх ёстой.

```
-- Серверийн Login үүсгэх (Mixed Mode идэвхтэй байх ёстой)
CREATE LOGIN db_admin
    WITH PASSWORD      = 'Admin@Secure2024',
         CHECK_POLICY  = ON,
         CHECK_EXPIRATION = ON;

CREATE LOGIN dept_reader
    WITH PASSWORD     = 'Reader@2024!',
         CHECK_POLICY = ON;

CREATE LOGIN student_writer
    WITH PASSWORD     = 'Writer@2024!',
         CHECK_POLICY = ON;

-- Үүсгэгдсэн Login-уудыг шалгах
SELECT name, type_desc, is_disabled, create_date
FROM   sys.server_principals
WHERE  type = 'S' AND name NOT LIKE '##%'
ORDER  BY create_date DESC;
```

1б. Өгөгдлийн сангийн User үүсгэж Login-тай холбох (0.4 оноо)

Үүсгэсэн Login тус бүрт UniversityDB дотор хэрэглэгч үүсгэж холбоно.

```
USE UniversityDB;
GO

-- Login-тай холбоотой User үүсгэх
CREATE USER db_admin       FOR LOGIN db_admin;
CREATE USER dept_reader    FOR LOGIN dept_reader;
CREATE USER student_writer FOR LOGIN student_writer;

-- Үүсгэгдсэн User-уудыг шалгах
SELECT name, type_desc, create_date
FROM   sys.database_principals
WHERE  type = 'S'
  AND  name NOT IN ('dbo','guest','INFORMATION_SCHEMA','sys')
ORDER  BY create_date DESC;
```

Login vs User тайлбар:
LOGIN нь серверийн түвшний нэвтрэх эрх — хаалгын түлхүүр. USER нь тухайн өгөгдлийн санд хандах эрх — өрөөний түлхүүр. Нэг LOGIN нь олон өгөгдлийн санд USER-тэй байж болно.

Даалгавар 2: Роль үүсгэж эрх олгох — RBAC | 0.8 оноо

2а. Роль үүсгэж GRANT хийх (0.4 оноо)

Гурван ялгаатай роль үүсгэж, тус бүрт тохирсон эрх олгон хэрэглэгчдийг хуваарилна.

```
USE UniversityDB;
GO

-- Роль үүсгэх
CREATE ROLE role_admin;           -- Бүх эрхтэй
CREATE ROLE role_dept_reader;     -- Зөвхөн Department унших
CREATE ROLE role_student_editor;  -- Student оруулах/засах эрхтэй

-- role_admin: бүх хүснэгтэд бүрэн эрх
GRANT SELECT, INSERT, UPDATE, DELETE ON Department TO role_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON Student    TO role_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON SecretData TO role_admin;

-- role_dept_reader: Department зөвхөн уншина
GRANT SELECT ON Department TO role_dept_reader;

-- role_student_editor: Student-д SELECT/INSERT/UPDATE
GRANT SELECT, INSERT, UPDATE ON Student TO role_student_editor;

-- Хэрэглэгчдийг роль руу нэмэх
ALTER ROLE role_admin          ADD MEMBER db_admin;
ALTER ROLE role_dept_reader    ADD MEMBER dept_reader;
ALTER ROLE role_student_editor ADD MEMBER student_writer;
```

2б. DENY ба REVOKE хэрэглэх (0.4 оноо)

dept_reader хэрэглэгчид SecretData хүснэгтэд хандахыг DENY-аар хатуу хориглоно. Дараа нь role_student_editor-ийн UPDATE эрхийг REVOKE хийж, ялгааг харуул.

```
-- DENY: dept_reader-ийг SecretData-аас бүрмөсөн хаах
DENY SELECT ON SecretData TO dept_reader;

-- REVOKE: role_student_editor-ийн UPDATE эрхийг цуцлах
REVOKE UPDATE ON Student FROM role_student_editor;

-- Одоогийн эрхийн байдлыг шалгах
SELECT
    dp.name                    AS Хэрэглэгч,
    OBJECT_NAME(p.major_id)    AS ОбьектНэр,
    p.permission_name          AS Эрх,
    p.state_desc               AS Төлөв
FROM sys.database_permissions p
JOIN sys.database_principals  dp
    ON p.grantee_principal_id = dp.principal_id
WHERE dp.name IN ('dept_reader','student_writer','db_admin',
                  'role_admin','role_dept_reader','role_student_editor')
ORDER BY dp.name, p.permission_name;
```

⚠ **DENY ба REVOKE-ийн ялгаа:**
REVOKE: олгосон эрхийг буцааж авна. Хэрэглэгч өөр роль эсвэл шууд GRANT-аар дахин эрх авч болно — зөөлөн хандлага. DENY: эрхийг бүрмөсөн хаана. GRANT байсан ч DENY нь давамгайлна. Хэрэглэгч ямар ч замаар эрх авч чадахгүй — хатуу хандлага.

Даалгавар 3: Хэрэглэгчийн эрхийг шалгах	**0.6 оноо**

3а. EXECUTE AS USER — Хэрэглэгч болж дуурайлах (0.4 оноо)

EXECUTE AS USER командаар хэрэглэгч болж дуурайлган, тухайн хэрэглэгчийн нүдэнд юу харагдахыг практикт шалгана.

```
USE UniversityDB;
GO

-- ============================================
-- dept_reader дуурайлах
-- ============================================
EXECUTE AS USER = 'dept_reader';
    SELECT * FROM Department;   -- Амжилттай (GRANT байгаа)
    SELECT * FROM SecretData;   -- АЛДАА! (DENY тавьсан)
REVERT;

-- ============================================
-- student_writer дуурайлах
-- ============================================
EXECUTE AS USER = 'student_writer';
    SELECT FirstName, LastName, GPA FROM Student;  -- Амжилттай

    INSERT INTO Student
        (FirstName, LastName, RegistrationNumber, Gender, DepartmentID, Status, GPA)
    VALUES (N'Туршилт', N'Хэрэглэгч', 'T999999', N'Эрэгтэй', 1, N'Идэвхтэй', 3.00);
    -- INSERT амжилттай (GRANT SELECT, INSERT байгаа)

    UPDATE Student SET GPA = 4.00 WHERE RegistrationNumber = 'T999999';
    -- UPDATE АЛДАА! (REVOKE хийсэн тул)
REVERT;

-- Туршилтын мөр цэвэрлэх
DELETE FROM Student WHERE RegistrationNumber = 'T999999';
```

3б. Бичгийн асуулт — RBAC тайлбар (0.2 оноо)

📝 **Дараах асуултуудад өөрийн үгээр хариулна уу (5-8 өгүүлбэр):**
• db_admin болон dept_reader хоёрын эрхийн ялгааг жишээтэйгээр тайлбарла. • DENY ба REVOKE-ийн ялгааг лабораторийн туршилт дотор юу харсан болохоор тайлбарла. • Хамгийн бага эрхийн зарчим (Least Privilege) яагаад чухал вэ? Байгаль орчны жишээ оруул.

Даалгавар 4: SQL Injection хамгаалалт — 0.4 оноо

4a. Аюултай ба аюулгүй Stored Procedure үүсгэж туршилт хийх (0.4 оноо)

SQL Injection халдлага хэрхэн ажилладгийг харуулж, параметрт асуулгаар хэрхэн хамгаалдагийг практикт туршина.

```
USE UniversityDB;
GO

-- ============================================
-- АЮУЛТАЙ: Динамик SQL — Injection-д нэрвэгдэнэ
-- ============================================
CREATE PROCEDURE sp_GetStudent_Unsafe
    @RegNum NVARCHAR(50)
AS
BEGIN
    DECLARE @SQL NVARCHAR(500);
    SET @SQL = N'SELECT FirstName, LastName, GPA FROM Student
                 WHERE RegistrationNumber = ''' + @RegNum + '''';
    EXEC sp_executesql @SQL;
END;
GO

-- Injection туршилт: бүх оюутан гарна (АЮУЛТАЙ!)
EXEC sp_GetStudent_Unsafe @RegNum = 'X'' OR ''1''=''1''--';

-- ============================================
-- АЮУЛГҮЙ: Параметрт асуулга
-- ============================================
CREATE PROCEDURE sp_GetStudent_Safe
    @RegNum NVARCHAR(50)
AS
BEGIN
    SELECT FirstName, LastName, GPA
    FROM   Student
    WHERE  RegistrationNumber = @RegNum;
END;
GO

-- Injection оролдлого: 0 мөр буцаана (аюулгүй!)
EXEC sp_GetStudent_Safe @RegNum = 'X'' OR ''1''=''1''--';

-- Зөв хэрэглэлт
EXEC sp_GetStudent_Safe @RegNum = 'B220001';
```

Дүгнэлт:
sp_GetStudent_Unsafe-д injection дамжуулахад БҮГД оюутны мэдээлэл гарна — аюулгүй байдлын ноцтой зөрчил! sp_GetStudent_Safe-д injection дамжуулахад 0 мөр гарна — параметр нь SQL-ийн хэсэг биш, өгөгдөл хэлбэрээр дамждаг учраас injection боломжгүй.

Даалгавар 5: SQL Server Audit тохируулах ба логуудыг шинжлэх — 0.4 оноо

5а. Server Audit үүсгэж идэвхжүүлэх (0.2 оноо)

UniversityDB-ийн Student хүснэгтэд хийгдсэн INSERT, UPDATE, DELETE үйлдлүүдийг аудитлах тохиргоо хийнэ.

```
USE master;
GO

-- Алхам 1: Server Audit үүсгэх
-- (Windows Explorer-т C:\AuditLogs\ хавтас байгаа байх ёстой)
CREATE SERVER AUDIT UniversityAudit
    TO FILE (
        FILEPATH          = 'C:\AuditLogs\',
        MAXSIZE           = 10 MB,
        MAX_FILES         = 5,
        RESERVE_DISK_SPACE = OFF
    )
    WITH (ON_FAILURE = CONTINUE);
GO

-- Алхам 2: Audit идэвхжүүлэх
ALTER SERVER AUDIT UniversityAudit WITH (STATE = ON);
GO

-- Алхам 3: DB-level Audit Specification үүсгэх
USE UniversityDB;
GO

CREATE DATABASE AUDIT SPECIFICATION StudentDataAudit
    FOR SERVER AUDIT UniversityAudit
    ADD (INSERT ON dbo.Student    BY [public]),
    ADD (UPDATE ON dbo.Student    BY [public]),
    ADD (DELETE ON dbo.Student    BY [public]),
    ADD (SELECT ON dbo.SecretData BY [public])
    WITH (STATE = ON);
GO

-- Шалгах: идэвхтэй аудитуудын жагсаалт
SELECT name, status_desc, audit_file_path
FROM   sys.server_audits;
```

5б. Аудит ажиллаж байгааг баталж лог унших (0.2 оноо)

Аудит ажиллаж байгааг баталгаажуулах зорилгоор Student хүснэгтэд зарим өөрчлөлт хийж, лог файлаас эдгээр үйлдлүүдийг хайж олно.

```
USE UniversityDB;
GO

-- Аудитлагдах үйлдлүүд хийх
INSERT INTO Student
    (FirstName, LastName, RegistrationNumber, Gender, DepartmentID, Status, GPA)
VALUES (N'Аудит', N'Тест', 'AUDIT001', N'Эрэгтэй', 1, N'Идэвхтэй', 3.50);

UPDATE Student SET GPA = 3.99 WHERE RegistrationNumber = 'AUDIT001';

DELETE FROM Student WHERE RegistrationNumber = 'AUDIT001';

-- ============================================
```

```
-- Аудитын лог файлаас унших
-- ============================================
SELECT
    event_time              AS Цаг,
    server_principal_name   AS Хэрэглэгч,
    database_name           AS ӨгөгдлийнСан,
    object_name             AS ХүснэгтНэр,
    action_id               AS Үйлдэл,
    succeeded               AS АмжилттайЭсэх,
    statement               AS SQLКод
FROM sys.fn_get_audit_file('C:\AuditLogs\*.sqlaudit', DEFAULT, DEFAULT)
ORDER BY event_time DESC;

-- Зөвхөн Student хүснэгттэй холбоотой лог шүүх
SELECT event_time, server_principal_name, action_id, statement
FROM   sys.fn_get_audit_file('C:\AuditLogs\*.sqlaudit', DEFAULT, DEFAULT)
WHERE  object_name = 'Student'
ORDER  BY event_time DESC;
```

Лабораторийн цэвэрлэгээ (дуусгасны дараа):

```
USE UniversityDB; ALTER DATABASE AUDIT SPECIFICATION StudentDataAudit
WITH (STATE = OFF); DROP DATABASE AUDIT SPECIFICATION StudentDataAudit;
DROP USER db_admin; DROP USER dept_reader; DROP USER student_writer;
USE master; ALTER SERVER AUDIT UniversityAudit WITH (STATE = OFF); DROP
SERVER AUDIT UniversityAudit; DROP LOGIN db_admin; DROP LOGIN
dept_reader; DROP LOGIN student_writer;
```

5 ҮНЭЛГЭЭНИЙ РУБРИК

Лабораторийн ажлыг дараах шалгуурын дагуу нийт 3.0 оноогоор үнэлнэ:

№	Даалгавар	Оноо	Үнэлгээний шалгуур
0	**Өгөгдлийн бэлтгэл**	0.0	UniversityDB-д SecretData хүснэгт үүссэн, өгөгдөл оруулагдсан
1	**Login & User үүсгэх**	0.8	3 LOGIN, 3 USER зөв үүсгэж UniversityDB-тэй холбосон
2	**Роль үүсгэж эрх олгох (RBAC)**	0.8	3 ROLE үүсгэж, GRANT/DENY/REVOKE зөв хэрэглэсэн
3	**Хэрэглэгчийн эрхийг шалгах**	0.6	EXECUTE AS USER ашиглан эрхийн ялгааг баталгаажуулсан
4	**SQL Injection хамгаалалт**	0.4	Аюултай болон аюулгүй SP хоёуланг туршиж ялгаа харуулсан
5	**Аудит тохируулж лог унших**	0.4	SERVER AUDIT үүсгэж, үйлдлүүд бүртгэгдэж, лог уншсан
НИЙТ ОНОО:		**3.0**	

Нэмэлт үнэлгээний шалгуур

- Кодын форматчлал: SQL код уншигдахуйц, монгол тайлбар бичигдсэн
- Injection туршилт: Аюултай болон аюулгүй SP-ийн ялгаа тодорхой харагдсан
- Бичгийн асуулт (3б): Эрхийн ялгааг жишээтэй тайлбарласан

- Аудитын лог: Screenshot хавсаргасан буюу .sql файл ирүүлсэн
- Цэвэрлэгээ: Туршилтын хэрэглэгч, роль, аудит объектуудыг устгасан

6 ХУГАЦАА БА ДҮРЭМ

- Лабораторийн ажлыг 15-р долоо хоногт багтан хамгаалах
- Бусдын ажлыг хуулбарласан тохиолдолд 0 оноо авна
- SQL Server-ийн аюулгүй байдлын тохиргоог серверийн зөвшөөрлөөр хийнэ

Амжилт хүсье!
Аюулгүй байдал бол нэг удаагийн тохиргоо биш — байнгын хяналт, шинэчлэлтийн үйл явц юм. Асуулт гарвал багш эсвэл туслах багштайгаа холбогдоно уу.